

The Closed Manufacture Loop

Receipt-Sealed Autonomy for a Self-Hosting Software Fabric

A thesis presented for the degree of Doctor of Philosophy
Design Science of Software Manufacture

Manufactured text — drafted by the fleet’s own coding engine from the
session receipts of 2026-09-26/27.

Human supervisor and examiner: S. Chatman.

The authorship split is itself Result 7.1.

DfCM Research Group (Design for Combinatorial Maximalism)

Calver edition v27.9.27 • one canonical checkout, no worktrees

2026-09-27

Abstract

Software engineering has spent sixty years driving human cost out of the *construction* of code. It has almost entirely failed to drive human cost out of the *admission* of code — the decision that a change is real, tested, authorized, and worth keeping. This thesis argues that these are not two problems but one, and that the second is the actual blocker to autonomy. We formalize software manufacture as a closed recursion over a typed observation space: raw observation O becomes admitted observation O^* only by passing an admission function that demands grounding (exact subject identity), bounds, and witnessed consequence; manufacture is a function μ lawful only on O^* ; and every actuation must return a receipt R carrying identity, authority, consequence, replay, and standing. We call the fixed point of this recursion the **closed manufacture loop**, and state its central invariant in system terms: a job that makes no progress for longer than a stated bound must be reclaimed, receipted, and routed around — never awaited by a human.

We validate the theory by construction and by falsification attempt. The artifact is a production fabric supervising a fleet worker CLI over a 149-repository fleet: an Oban-cron-driven wave loop that claims work orders, binds them to per-epoch leases behind a fail-closed admission gate, executes real worker turns, seals head-verified receipts, exports process events in OCEL 2.0 form, and settles a machine-readable plan file — with no human on any execution edge. In the validation window the fabric (a) completed a 14-step burn-in autonomously after a five-defect causal chain was root-caused and each defect converted to a permanent guard, (b) executed an 11-wide concurrent inventory sweep of the entire fleet through a read-only gate profile, producing 158 evidence-bearing rows and a measured provider-throttling profile (43 failed model requests, bursts of 16/min, zero worker deaths), and (c) dogfooded a repair work order onto itself and *honestly failed acceptance* — the patch was reverted by its own coordinator, and the failure was kept as typed capital. Three of the author-engine’s own empirical claims were falsified during the window and are reported here with their corrections, because a fabric that cannot falsify its coordinator cannot falsify anything.

The thesis contributes: (1) a type-theoretic reading of agent accountability in which “the workflow ran” is a category error; (2) a failure taxonomy that maps every observed defect

to exactly one broken type term; (3) the wedge-and-reclaim invariant as a design pattern for work-conserving autonomy; (4) a measured account of provider throttling as a *width* signal rather than a stop condition; and (5) a retirement metric for human supervision, reported as the count of things the human did not have to write. We conclude that the loop can be closed not by making the generator stronger, but by making refusal cheap, receipts mandatory, and honesty mechanical.

Contents

1	Introduction: The Admission Problem	5
1.1	The wrong bottleneck	5
1.2	The retirement thesis	5
1.3	Research questions	5
1.4	Method in one paragraph	6
2	Background and Related Work	6
3	Theory: The Chatman Equation as a Type System for Agency	7
3.1	The equation	7
3.2	The recursion	7
3.3	The failure taxonomy	7
3.4	The wedge-and-reclaim invariant	8
4	The Artifact: A Fabric That Manufactures Itself	8
4.1	Topology	8
4.2	The wave loop	9
4.3	The gate	9
4.4	Identity: per-epoch leases	9
5	Method: Falsifier-Driven Development	9
6	Evaluation: One Night as a Series of Experiments	10
6.1	E1 — The burn-in: closing the loop against five defects	10
6.2	E2 — The fleet sweep: width, throttling, and a read-only authority	11
6.3	E3 — The dogfood cycle: the fabric repairing its own producer	11
6.4	E4 — The epistemology of measurement	12
6.5	E5 — The wedge: 80 minutes of <code>executing</code>	12
7	Discussion: What Closed, What Remains Open	12
7.1	RQ1 — closure	12
7.2	RQ2 — the taxonomy earns its keep	12
7.3	RQ3 — falsification is the cheapest QA	13
7.4	RQ4 — width is a control variable	13
7.5	RQ5 — the gate is the constitution	13
7.6	Retirement accounting (the ∂ metric)	13

8	Limitations and Threats to Validity	13
9	Future Work: The Capital Census	14
10	Conclusion	14
A	The Failure Taxonomy, Instantiated	14
B	Receipt Ledger (window numbers)	15

1 Introduction: The Admission Problem

1.1 The wrong bottleneck

The last decade made code generation cheap. Benchmarks parade; vendors ship; a single model call produces a plausible module in seconds. Yet the cost of shipping software has not fallen by two orders of magnitude, and the reason is not mysterious: generation was never the expensive part. The expensive part is **admission** — the human-gated decision that a generated change is (i) what it claims to be, (ii) tested against reality rather than against its own optimism, (iii) authorized to touch what it touched, and (iv) worth keeping given everything else that exists. Code review, CI triage, release management, incident response: these are admission rituals, and they are staffed by humans because until now no machine artifact could carry enough *accountability structure* to be admitted without one.

This thesis takes the position that accountability structure is a *property of the artifact*, not of the process surrounding it. If every change carries its own proof — identity (which exact bytes, on which exact subject), authority (under which lease, within which fence), consequence (what durably changed), replay (can a stranger re-execute the proof), and standing (what did we actually observe, in the standing vocabulary `ALIVE` / `PARTIAL_ALIVE` / `BLOCKED` / `REFUSED_*` / `UNKNOWN` / `UNSUPPORTED`) — then admission becomes a mechanical check, and the human edge can be removed without removing the safety. If it does not carry that structure, no amount of review theatre makes it trustworthy.

1.2 The retirement thesis

We adopt as a design objective the operator’s law:

$$\partial \text{Outcome} / \partial \text{ExceptionalHuman} \rightarrow 0.$$

The marginal contribution of exceptional human judgment to the outcome should fall toward zero — not by deskilling the human, but by *externalizing* every recurring judgment into ontology, schema, generator, verifier, or process control, so the same reasoning is never purchased twice. The unit of progress is not features shipped; it is the count of judgments that used to require a person and no longer do. Chapter 7 reports this count for the validation window.

1.3 Research questions

- **RQ1.** Can the observe → admit → execute → receipt → settle recursion be closed in a production software fleet, with zero human edges on execution?
- **RQ2.** What is the failure taxonomy of a system that types its claims — i.e., when the loop fails, does it fail at a *nameable term*, and does that name determine the repair?
- **RQ3.** Is adversarial self-falsification (courts, skeptics, revert-mutation) mechanically useful, or merely ritual?
- **RQ4.** How should a work-conserving fabric respond to provider throttling at concurrency — as a stop condition or as a control signal?
- **RQ5.** What survives of the generator’s authority when admission is separated from execution by a fail-closed gate?

1.4 Method in one paragraph

Design science: build the artifact to the theory, run it in production conditions, and treat every failure as an experiment whose result is a type term plus a permanent guard. The validation window is one continuous operational period (2026-09-26T21:50Z through 2026-09-27T19:05Z) during which the fabric executed a 14-step burn-in plan, an 11-wide fleet-wide sweep, and a self-repair work order — 176 loop ticks recorded, 66 exported process events validated against the OCEL 2.0 conformance court, and every number in this thesis traceable to a receipt, a log, or a query in the canonical checkouts.

2 Background and Related Work

Autonomic computing. Kephart and Chess’s MAPE-K loop — Monitor, Analyze, Plan, Execute over shared Knowledge [1] — is the ancestral form of our recursion. Our contribution is to type the loop’s edges: monitoring produces raw O , but *planning* may only consume O^* ; execution is μ ; and the Knowledge component is not a database but the receipt ledger, with standing derived from receipts rather than asserted. Self-adaptive software research [2, 3] offers control-theoretic formulations of adaptation; we add an *economics of honesty*: an adaptation that cannot be falsified after the fact has zero evidential value, whatever its control-loop elegance.

Process mining. The fabric exports every execution as object-centric event logs conforming to OCEL 2.0 [4], validated by a production conformance task, not by inspection. This grounds the loop’s self-description: the thesis’s numbers are re-derivable from the event logs by a stranger, which is the reproducibility standard we take from process mining and give to agent fleets.

Supply-chain provenance. in-toto [5] and SLSA [6] established that software artifacts should carry signed attestations about *how they were produced*. Our receipts are the same move applied at commit granularity and consumed *inside* the producing system: the loop’s next observation admits only receipt-backed standing. The R-missing taxonomy of Chapter 3 is, in their language, a schema for attestation incompleteness — with the difference that a missing field is not merely non-compliant but *semantically typed* (identity missing $\neq$ authority missing $\neq$ replay missing), and each type names its own repair.

Mutation testing and anti-vacuity. DeMillo, Lipton and Sayward’s mutation hypothesis [7] holds that tests are adequate when they kill mutants. We generalize: *every* claimed proof-carrying behavior in the loop must be paired with the observation that kills it — a gate must have a witnessed firing, an acceptance criterion must fail on the reverted subject, an autonomy claim must have a cron row in the database. During the window this principle killed one coordinator claim (that the loop ran autonomously) via a two-row database query, which is the cheapest falsifier we know.

Planning and FOND execution. The wave loop is a semi-structured execution of a plan encoded in a plain markdown table (step, status, evidence) — deliberately not PDDL, though the fleet also carries FOND planners [8]. The lesson we confirm for agent systems: the machine-readable plan is only worth what its *settlement* machinery enforces; a plan file no one is forced to update is fiction with a syntax.

Code generation and its discontents. Brooks [9] set the irreducible essence of software at complexity, conformity, changeability, invisibility. This thesis is, in a narrow sense, an attack on the conformity axis: the fabric’s admission structure forces generated change to conform

to typed evidence rather than to reviewer judgment. We claim no silver bullet — Chapter 8 documents what remained stubbornly human.

3 Theory: The Chatman Equation as a Type System for Agency

3.1 The equation

The operator’s equation is deceptively small:

$$A = \mu(O^*) \quad R = \text{receipt}(A)$$

Read as types:

- O — raw observation: any claim about the world. A survey result, a test summary in a chat message, a commit message, a model’s confident sentence. **All of it is O .**
- O^* — admitted observation: O that is aligned, grounded (exact subject identity — the specific SHA, the specific log file), bounded, and admitted by a function that can *refuse*. The admission function is not a filter; it is a gate with an anti-vacuity property: a gate that never refuses is not a gate (**admission_vacuous**), it is the identity function wearing a uniform.
- μ — lawful manufacture. A function from O^* to actuation. $\mu(O)$ is a **type error** — the category error that underlies most agent misbehavior: acting on an unadmitted claim.
- R — the receipt of actuation: five fields, {identity, authority, consequence, replay, standing}. A report missing a field is not a weak receipt; it is *not a receipt* (**R_missing_***).

3.2 The recursion

$$A_t = \mu(O_t^*); \quad R_t = \text{receipt}(A_t); \quad O_{t+1}^* = \text{admit}(\text{observe}(w(A_t)) \cup R_t).$$

Autonomy is the closure of this recursion with no human on any edge. Two consequences follow immediately and are tested empirically in Chapter 6:

1. **Autonomy is an edge product**, not a node property: $\text{Autonomy} \approx \prod_{\text{edges}} \text{standing}$. A chain of nine perfect nodes and one human handoff has $\text{autonomy} \approx 0$. The burn-in’s steps-1–9 period had eight perfect edges and one false claim about the tenth; the product was zero, and only the falsification of that claim re-opened the path to closure (§6.1).
2. **Standing is derived, never stored as truth**. A stored “ALIVE” is a cache without invalidation. The loop’s own telemetry — 176 appended lines, each a settled outcome — is the standing derivation log; any state that cannot point at its line is UNKNOWN by construction.

3.3 The failure taxonomy

Every failure observed in the window types to exactly one broken term. The terms and their instantiations (full table in Appendix A): **mu_on_0** (dispatching against a retired checkout; “cron

ran autonomously” asserted from a dashboard glance); `admission_vacuous` (a token-length check that passed `Bearer undefined`); `R_missing_identity` (a claim citing another tree’s scratch file); `R_missing_consequence` (13 ticks that produced telemetry but no cron row — “the workflow ran” $\neq$ “the loop ran”); `R_missing_replay` (receipts that lived only in a driver’s ephemeral BEAM); `R_not_fed_back` (steps 10–14 pending for hours because no one read the queue’s two-row history).

The taxonomy’s practical power is repair-routing: `mu_on_0` $\rightarrow$ re-ground the observation; `admission_vacuous` $\rightarrow$ build the anti-vacuity witness; `R_missing_replay` $\rightarrow$ move the proof into the durable ledger. During the window, repair time correlated strongly with how early the failure got its type name.

3.4 The wedge-and-reclaim invariant

The one *new* theoretical contribution of the validation window, extracted from an 80-minute operational failure and stated as a system law:

$$\forall \text{job} : \text{no_progress}(\text{job}, t > \text{bound}) \Rightarrow \text{reclaim}(\text{job}) \wedge \text{receipt}(\text{job}) \wedge \text{continue}(\text{next}).$$

In any work-conserving fabric, the worst failure mode is not crash and not wrong output — both are visible. It is the *typed-unbounded wait*: a work item whose state says `executing` while its executor is dead. This state is invisible to every health check that asks “is it running?” and visible to every check that asks “is it progressing?” The invariant demands three things: a bound per phase, reclamation that releases the resource (the lease, the provider slot), and a receipt for the reclamation itself — so the fabric’s next observation admits the truth. Section 6.5 shows the failure that taught it and the two-layer guard that encodes it.

4 The Artifact: A Fabric That Manufactures Itself

4.1 Topology

Two canonical checkouts, one fleet:

- **xaas** — the fabric. An Elixir/Phoenix application whose center is `Xaas.Ultracode`: a run/epoch/lease/receipt model on Postgres, an Oban job queue whose crontab drives the wave loop every five minutes, a provider pool fencing live leases per provider (capacity 10), a dispatch layer that spawns real worker processes with a per-epoch lease identity, and an OCEL 2.0 egress that exports every run as a process-mining event log validated by a conformance task.
- **zcode-cli** — the worker. A thin CLI/TUI around a vendored agent runtime; its `gall-work` module implements the lease lifecycle (claim $\rightarrow$ construct $\rightarrow$ close) against the fabric’s MCP verbs (`claim_next`, `admit_tool`, `heartbeat`, `close_candidate`), behind a `PreToolUse` gate that *denies or defers, never grants, and fails closed*.

The separator between them is the thesis in miniature: the worker may execute; only the gate admits; only receipts settle.

4.2 The wave loop

The loop consumes a plan file (`STATE.md`) whose table is `step status | evidence|` and whose dependency chain is a `REMAINING` note. Per tick: parse (typed refusal on corruption, never a crash) → select the first owing step not held by a live lease → construct a work order bound to an epoch → dispatch one real worker synchronously → settle the row from the *sealed receipt's* verdict, never from the exit code alone → append one telemetry line. The plan file is therefore not documentation; it is the loop's memory, updated only by machinery that saw a receipt.

4.3 The gate

The admission gate is a `PreToolUse` hook evaluated on every consequential tool call of a leased worker. Its design properties: default-deny with an explicit allowlist; worktree containment for writes; sensitive-directory denial even in widened read modes; **typed, evidence-bearing refusals** (a denial names what was refused and why, so the worker — and the audit log — learn); and the anti-vacuity discipline that every widening (the keyed lease read; the read-only sweep profile) is witnessed by allow *and* deny probes before it is trusted. The gate's refusals cost the fabric two work sessions in the window and were correct both times — once against a worker whose fabric token was broken, once against a work order whose boundary the coordinator had drawn wrongly. A gate that never annoys you is not enforcing anything.

4.4 Identity: per-epoch leases

Concurrency first failed here: two workers sharing one surface collided on a single per-cwd lease file (exit 65, a lawful conflict). The repair — suffix the lease state path with the epoch id, propagated end-to-end from dispatcher env through worker to gate read — is trivial as engineering and deep as theory: **identity is the field without which no other receipt field means anything**. The sweep then ran 11 workers against one surface with zero collisions.

5 Method: Falsifier-Driven Development

Five rules governed the window; each earns its place by a failure it prevented or caught.

1. **Fan-out first.** Deliberation is not a sequential phase; on any multi-part task, launch the concurrent wave immediately and let thinking be distributed. The window ran three waves (10-lane wave 1; a resumed half-wave; the fabric's own 11-wide sweep). Collisions were permitted by doctrine — overlapping work resolves at integration toward the stronger implementation, and the dispute is settled by evidence, not seniority.
2. **Courts kill, coordinators integrate.** The coordinator's sequential role shrinks to: write the contract, resolve collisions on concrete artifacts, run the courts. A dedicated skeptic lane re-derived the window's headline numbers independently and refuted two of the coordinator's claims (§6.4). The falsifier: any defect caught by coordinator pre-thinking rather than a court marks the wave as decoration.
3. **Cheapest high-information gate first.** Two database queries falsified the autonomy claim that a dashboard had “confirmed.” Unit before integration before e2e — and

when a real-provider e2e fails identically on the pre-change commit, the bisect costs four commands and saves a night of chasing a ghost.

4. **Repair leaves capital.** Every failure must exit as one of: code, regression test, typed error, watchdog, schema, or receipt. The 80-minute wedge exited as a config line (Lifeline), a law (wedge-and-reclaim), and a test; the gate key-mismatch exited as a witness-probe triplet (allow/deny/deny).
5. **No fake success.** Planned $\neq$ implemented $\neq$ committed $\neq$ pushed $\neq$ merged $\neq$ observed. The vocabulary is short on purpose. Where the window could not observe (the live-provider test; the cold-replay toolchain), the record says deferred or UNKNOWN — the standing vocabulary’s empty slots are information.

6 Evaluation: One Night as a Series of Experiments

6.1 E1 — The burn-in: closing the loop against five defects

Design. 14 real steps in a scratch git repository, one real worker turn per step, dispatched by the loop, settled only on head-verified receipts, with OCEL 2.0 validation over every run.

Result. 14/14 steps DONE; 52/52 runs OCEL-valid; the loop recorded :complete and continued ticking (176 telemetry lines at window end) with no human in any execution edge. Steps 10–14 were completed by cron-driven ticks 20–23 *after* the human’s driver BEAM had exited — the temporal signature of genuine closure.

The five-defect causal chain. The window’s pedagogical core is that the burn-in initially failed through five *independent* defects stacked in series, each masked by the next:

1. **R_missing_replay** (NextEpoch worktree drop). Tick-constructed epochs lost the carried worktree; workers landed in a fence with no surface. Fixed with a one-line carry-through plus regression test.
2. **mu_on_0** (stale dispatch target). Dispatch pointed at a retired checkout. Fixed by retiring the checkout with a rollback-qualified archive and repointing to the canonical one.
3. **R_missing_consequence** (gate manifest + token interpolation). The gate could not resolve the fabric endpoint from the plugin manifest, and the coordinator’s vacuous check passed **Bearer undefined**. Both fixed; the vacuity was caught by switching from length-checks to strict equality — **admission_vacuous** dies only when the gate is forced to refuse.
4. Fence/scope mismatch. Workers were ordered to edit files outside their fence; the gate refused; the goal text was repaired to respect the boundary — the fabric teaching its coordinator.
5. The claim defect (**R_missing_consequence**, coordinator-grade). The previous session’s receipt claimed “the 5-minute cron continued autonomously.” The database said otherwise: the queue held two rows *ever*, one a five-day-old zombie in **executing**, because the worker’s infinite-period uniqueness dropped every cron enqueue. **A 2-row query falsified a receipt.** This is RQ3 answered affirmatively with the cheapest possible instrument.

The completion mechanics. With the zombie cancelled and the token parity restored, ticks 20–23 completed steps 10–14 in clean 5-minute cadence, each closing `partial_alive`, `head_verified=true`. The distinguishing observation between “automation” and “autonomy” is precisely that the last five steps ran while every human-owned process from earlier that night was dead.

6.2 E2 — The fleet sweep: width, throttling, and a read-only authority

Design. The operator ordered the entire fleet searched — then corrected the mechanism: *through the fabric, at width*. 149 repositories were batched into 11 work orders, bound to a single shared work surface, dispatched concurrently through the real `Dispatch` boundary with per-epoch lease identities and a new opt-in **read-only sweep profile** in the gate (git read-only subcommands and read helpers may leave the worktree; sensitive directories stay denied; writes stay fenced; witnessed by `allow/deny/deny` probes before first use).

Result. 11/11 work orders completed with reports (158 repo rows: 48 `CURRENT` / 76 `DRIFTED` / 8 `UNKNOWN`), and 14/14 runs (including 3 half-pace retries) exported **OCEL 2.0-valid** event logs. Zero lease collisions on the shared surface — the per-epoch identity did its job at width.

Measured throttling (RQ4). The 11-wide wave met provider rate limits head-on. The honest instrument — structured `model_request_failed` events, not substring greps — recorded **43 failures, two bursts (16/min and 10/min), zero worker deaths**, because client retries absorb single events. The operator’s instruction — *get the stats instead of guessing* — converted a folklore (“we hit limits”) into a control signal: width should follow measured pressure. That signal is now code: adaptive dispatch width subtracts one per in-window signature and recovers as they age out. The half-pace retry of the three failed batches completed 3/3 with zero failures — the measured behavior the control law encodes.

6.3 E3 — The dogfood cycle: the fabric repairing its own producer

The highest-leverage defect found while consuming the sweep was the fabric’s own cold-replay toolchain resolution (a fresh `work_dir` carries no `.tool-versions`; the `asdf` shim then resolves a global toolchain whose beams mismatch the checkout — the “corrupt atom table” family, nine red tests). Per the self-hosting thesis, the repair was *filed as a work order and executed by a worker through the same claim/gate/receipt boundary*.

Result — a triple lesson. (i) The worker returned a complete, minimal, well-argued patch — generation is solved. (ii) The worker’s report is a model of typed honesty: standing **BLOCKED**, because the gate lawfully refused every edit outside its lease surface — the coordinator had drawn the work order’s authority boundary wrong; the gate caught its own coordinator, which is RQ5 answered: **separating admission from execution preserves generator authority precisely where it should — at the fence**. (iii) After harvest, the patch *failed acceptance* (corruption persisted; three prior repairs were each falsified in turn) and was **reverted by the coordinator it was offered to**. No fake success is not a slogan; it is a revert.

6.4 E4 — The epistemology of measurement

Three instrumentation errors were made and caught in the window, and each is a thesis-grade parable:

1. **The pipe that ate exit codes (twice).** `cmd | tail` reports *tail's* exit status; two separate “the validator is broken/fail-open” conclusions were artifacts of this. Repair: `cmd > file; echo $?` — and a standing suspicion of any pipeline whose status you quote.
2. **The substring grep that invented 500 events.** Counting 429 occurrences in raw logs matched echoed context text; the structured event count was 43. The instrument created the storm it reported.
3. **The dashboard that confirmed an autonomy claim.** Telemetry lines prove *ticks*; only the queue’s row history proves *who triggered them*. Two rows ever $\Rightarrow$ the cron never fired $\Rightarrow$ the claim was `mu_on_0`.

The general law: **an instrument is part of the observation, and *O* inherits its failure modes.** A fabric that quotes numbers must quote the query that produced them.

6.5 E5 — The wedge: 80 minutes of executing

An Oban job held the concurrency-1 loop queue for 80 minutes with no worker spawned and no settlement. Root cause chain, established from the database after the fact: the coordinator killed the server BEAM mid-tick (to restore a broken auth token — itself a defect); the executing row outlived its executor; **no Lifeline plugin was configured**, so the orphan waited forever, and recovery had required manual SQL — the exact anti-pattern the doctrine forbids. Repairs: `Oban.Lifeline` configured above the maximum legitimate tick; the wedge-and-reclaim invariant of §3.4 encoded as adaptive width + chaining + typed settle paths; and a SIGKILL regression test proving a dead worker’s row settles typed and the epoch reclaims without human SQL. The wedge taught the thesis’s best systems sentence: *the dangerous state is not failing — it is executing while dead.*

7 Discussion: What Closed, What Remains Open

7.1 RQ1 — closure

Closed for execution edges: observe (telemetry, queue rows) $\rightarrow$ admit (gate) $\rightarrow$ execute (real workers) $\rightarrow$ receipt (head-verified, sealed) $\rightarrow$ settle (STATE + standing). Open edges, honestly: **publication** (push/merge remained operator-gated), **capital** (what work is worth doing remains the operator’s census), and **cross-run memory** beyond receipts (the ontology layer). The loop’s autonomy is therefore real but *scoped*: the factory runs; the board of directors is still human. We consider this the correct shape — the human edges that remain are decisions about value and authority, not observations about truth.

7.2 RQ2 — the taxonomy earns its keep

Every one of the window’s ~ 15 distinct failures typed cleanly, and the type named the repair class in every case. We did not encounter a failure requiring two type names — encouraging

evidence the taxonomy is carved at the joints.

7.3 RQ3 — falsification is the cheapest QA

The autonomy claim fell to a 2-row query; the throttle folklore fell to a structured-event count; a live test’s “regression” fell to a 4-command bisect. Total cost of all three falsifications: minutes. Total cost of the beliefs had they persisted: a night each.

7.4 RQ4 — width is a control variable

Provider throttling presented as a *measurable pressure function over concurrency*, absorbed by retries at the event level. Treating it as stop/jam yielded three lost batches; treating it as width feedback (measured, then coded) yielded a control law with automatic recovery. We expect this to generalize to any resource whose failure mode is graceful degradation under multiplexing.

7.5 RQ5 — the gate is the constitution

The gate refused the correct things for the correct reasons, including against its own coordinator’s work order. Its costs (two lost work sessions, some conceptual friction) are the visible price of `admission_vacuous` never being allowed to return.

7.6 Retirement accounting (the ∂ metric)

In the window, the human did **not** write: the burn-in steps 10–14 and their commits; the entire fleet sweep’s 158-row inventory; the wave-loop settlement logic, gate widenings, and their witness probes; the P0 watchdog/lifeline/adaptive-width code and its 47-test suite; the census modules; the ticket ledgers. The human did write (or cause): the doctrine, the orders, the go/no-go on publication, and the capital census still pending. That ratio is the thesis’s empirical claim about $\partial \text{Outcome} / \partial \text{ExceptionalHuman}$ trending the right way — with the honest caveat that *trending* is all it is; the denominator still includes every strategy decision.

8 Limitations and Threats to Validity

1. $n = 1$ fabric, 1 night. The window is a strong existence proof, not a longitudinal study. The loop has now been observed across ~ 176 ticks and 66 OCEL-validated runs, but seasonality (provider load, fleet composition) is unmeasured.
2. The evaluator is inside the system. This thesis was drafted by the engine it evaluates, from its own receipts. The mitigation is that every number cites a query or artifact a stranger can re-execute (OCEL exports, DB rows, git SHAs) — but the *selection* of what to report is still self-selected. The operator’s examiner role is the external check.
3. Determinism of workers is unproven. The fleet sweep read-only surface is the only workload proven at width; write-heavy parallel work on shared surfaces awaits per-step work surfaces (git index contention is the known fence).

4. Environmental residuals. The cold-replay toolchain defect remains open after three falsified repairs; the live-provider lifecycle qualification remains red on a pre-wave commit (environmental, bisected). We report both as UNKNOWN rather than burying them.
5. Single-operator ontology. The standing vocabulary, the equation, and the doctrine are one operator’s formalism; transfer to a second organization is untested by construction.

9 Future Work: The Capital Census

The loop closes execution but opens an economics problem: with manufacture this cheap, *choosing what to manufacture* is the scarce act. The next architecture — already specified as a directive — inverts dispatch: a fleet receives a task, computes what it already knows how to do, and hands the worker engine **only the irreducible residual**: `WorkOrder` $\rightarrow$ `Bind` $\rightarrow$ `CapitalCensus` $\rightarrow$ `Applicability` $\rightarrow$ `Closure` $\rightarrow$ `ResidualGap` $\rightarrow$ `Route` $\rightarrow$ `Execute` $\rightarrow$ `Verify` $\rightarrow$ `Experience` $\rightarrow$ `Promote`, with a route lattice that prefers `reuse` $\prec$ `compose` $\prec$ `rule` $\prec$ `plan` $\prec$ `constraint` $\prec$ `generate` $\prec$ `specialized_model` $\prec$ `llm`, a `CapitalReceipt` whose fields strictly narrow (found $\neq$ applicable $\neq$ admitted $\neq$ sufficient $\neq$ authorized), and a retirement index $IEC_{n+1} = IEC_n - \text{Promoted}(C_n)$ whose fall *is* the $\partial \text{Outcome} / \partial \text{ExceptionalHuman}$ metric, measured. Two laws from the window seed it: machine-experience tuples (state, action, observation, outcome, evidence) make `RepeatedFailureReasoning` $> 1 \Rightarrow$ `SystemDefect` checkable; and the sweep reports are the first demand corpus the census will consume. Alongside: per-step work surfaces for write-heavy width; publication automation under explicit authority; and the standing fleet upgrades (76 DRIFTED rows) as the first census-demanded work orders.

10 Conclusion

The thesis set out to close a loop and found that closing it was mostly an exercise in *typing*: distinguishing what was observed from what was admitted, what ran from what settled, what was measured from what was grepped, what was refused from what was lost. The artifact now runs unattended through the night over a real fleet, converts its failures into typed capital, measures its provider instead of fearing it, and files repair orders against itself. Its coordinator reverted his own harvest when acceptance failed — and that revert, more than any benchmark, is the thesis’s result. A manufacture system is autonomous exactly to the degree that it cannot lie, cannot wait, and cannot waste: the receipts forbid the first, the watchdog forbids the second, and the census is how the third gets its turn.

A The Failure Taxonomy, Instantiated

#	broken term	observed as	repair that closed it
1	<code>R_missing_replay</code>	tick-constructed epochs lost the carried worktree	carry-through + regression test
2	<code>mu_on_0</code>	dispatch targeted a retired checkout	archive + repoint to canonical
3	<code>admission_vacuous</code>	token check passed <code>Bearer</code> undefined	strict-equality probe; evidence-bearing denials

4	fence/scope mismatch	workers ordered outside their fence	goal text respects the boundary; XAAS_SWEEP typed profile
5	R_missing_consequence	“ error ran autonomously” with 2 queue rows ever	zombie cancel + Lifeline; queue history is the falsifier
6	identity divergence	gate read the lease path the worker no longer wrote	keyed reads; witness probe triplet
7	executing-while-dead	80-min wedge after BEAM kill	Oban.Lifeline + wedge-and-reclaim invariant + SIGKILL test
8	instrument = O	pipe-eaten exit codes $\times 2$; substring-grep storm	file-redirect verification; structured-event counting
9	boundary design error	work order fenced away from its target	next orders bind lease worktree = target repo
10	acceptance failure	dogfood patch reverted on red tests	revert; residual typed UNKNOWN with falsified-fix ledger

B Receipt Ledger (window numbers)

- Loop ticks recorded: **176** (last: 19:05:00Z, outcome **complete**)
- Burn-in: **14/14 steps DONE**, closure by ticks 20–23, all head-verified
- OCEL v2: **52/52** burn-in runs valid; **14/14** fleet-sweep runs valid; **66** total runs validated across both
- Fleet sweep: 11 concurrent workers + 3 half-pace retries; **158 inventory rows** (48 CURRENT / 76 DRIFTED / 8 UNKNOWN); provider failures: **43 model_request_failed** (bursts 16/min, 10/min), retries absorbed 100%
- Gate witness probes: keyed lease (allow/deny/deny); sweep profile (allow/allow/deny-commit/deny-sensitive/deny-nosweep)
- Tests: wave_loop **47/47**; census+dispatch **85/85**; deterministic permission qualification **139 ms**; known environmental reds: 12 (typed, not from this wave)
- Commits (window, both repos): 20; branches: feat/alooop-lane2-autonomic-loop (xaas, base f1d42eb), fix/v26926-preview-publish-typed-skip (zcode-cli); unpushed — publication pending authority
- Human keystrokes in the window: doctrine citations, four orders (finish all work; upgrade toolchain; search all projects; route through ultracode), one stats demand, one factory prompt, one “status”, one thesis order

References

- [1] J. O. Kephart and D. M. Chess, “The Vision of Autonomic Computing,” *IEEE Computer*, 36(1), 2003.
- [2] M. Salehie and L. Tahvildari, “Self-adaptive software: Landscape and research challenges,” *ACM Trans. Auton. Adapt. Syst.*, 4(2), 2009.

- [3] D. Weyns, *An Introduction to Self-Adaptive Systems*, Wiley, 2020.
- [4] W. M. P. van der Aalst and D. Fahland (eds.), *Object-Centric Process Mining: OCEL 2.0 Standard*, ocel-standard.org, 2024.
- [5] S. Torres-Arias et al., “in-toto: Providing farm-to-table guarantees for bits and bytes,” *USENIX Security*, 2019.
- [6] Google et al., *SLSA: Supply-chain Levels for Software Artifacts*, slsa.dev, 2023.
- [7] R. A. DeMillo, R. J. Lipton, F. G. Sayward, “Hints on Test Data Selection: Help for the Practicing Programmer,” *IEEE Computer*, 11(4), 1978.
- [8] M. Ghallab, D. Nau, P. Traverso, *Automated Planning: Theory and Practice*, Morgan Kaufmann, 2004.
- [9] F. P. Brooks, Jr., “No Silver Bullet: Essence and Accidents of Software Engineering,” *IEEE Computer*, 20(4), 1987.
- [10] L. Lamport, *Specifying Systems: The TLA+ Language and Tools*, Addison-Wesley, 2002.
- [11] D. L. Parnas, “On the Criteria To Be Used in Decomposing Systems into Modules,” *CACM*, 15(12), 1972.
- [12] A. Cockburn, *Writing Effective Use Cases*, Addison-Wesley, 2001.

Defended by the receipts. What the examiner wrote: the doctrine, the orders, and this line.